

AI-Usage

I have pasted the prompt and the answer I have got directly from the AI.

For this task {i pasted the lexer part of the question}, I want you to give me a function skeleton code with todos to help me write the code (make sure not to provide any code inside the functions)

Here's the skeleton with function signatures in their original order, plus TODOs to guide you through implementing each one:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "lexer.h"

// ---- Check functions ----

static int is_special(char c){
    // TODO: return 1 if c is one of | & > < ; else return 0
}

static int is_space(char c){
    // TODO: return 1 if c is space, tab, \n, or \r, else 0
}

// ---- Linked list helpers ----

void tokenlist_init(TokenList *list){
    // TODO: set head/tail to NULL, count to 0
}

void tokenlist_free(TokenList *list){
```

```

    // TODO: walk the list freeing each token's value and the token itself
    // TODO: reset head/tail to NULL and count to 0
}

static int tokenlist_append(TokenList *list, TokenType type, char *value){
    // TODO: malloc a new Token, check for failure
    // TODO: set its type/value/next
    // TODO: link it onto list->tail (or set as head if list is empty)
    // TODO: increment list->count, return 0 on success / -1 on failure
}

// ---- Error message ----

static void lex_error(void){
    // TODO: print an "invalid syntax" message
}

// ---- Buf (dynamic char buffer) ----
// typedef struct { char *data; int len; int cap; } Buf;

static void buf_init(Buf *b){
    // TODO: set initial cap (e.g. 32), len = 0
    // TODO: malloc data, null-terminate if allocation succeeded
}

static int buf_push(Buf *b, char c){
    // TODO: bail out if b->data is NULL
    // TODO: grow (realloc, e.g. double cap) if len+1 would exceed cap
    // TODO: append c, null-terminate, increment len
    // TODO: return 0 on success / -1 on failure
}

// ---- Word reading ----

static int read_word(const char* line, int n, int *i, Buf *buf){
    // TODO: loop while *i < n
    // TODO: stop (break) if current char is space or special
    // TODO: handle '\\' escape: push next char, advance i by 2
    // (return -1 if backslash is the last char)
    // TODO: handle '"' quoting:
    // - advance past opening quote

```

```

//      - loop until closing '"', handling \" and \\ specially,
//      and other \x sequences by keeping the backslash
//      - return -1 if never closed
//      - advance past closing quote
//  TODO: handle '\"' quoting:
//      - advance past opening quote
//      - copy everything literally until closing '\"'
//      - return -1 if never closed
//      - advance past closing quote
//  TODO: otherwise just push the char and advance i
//  TODO: return 0 on success
}

// ---- Main entry point ----

int lex_line(const char* line, TokenList* out){
    // TODO: tokenlist_init(out)
    // TODO: loop over the line by index i:
    //      - skip spaces
    //      - match single-char tokens: | & ;
    //      - match > vs >> (lookahead)
    //      - otherwise: start a Buf, call read_word, append TOK_WORD
    //      (clean up and call lex_error on failure)
    // TODO: append a TOK_EOF token at the end
    // TODO: return 0 on success
    // TODO: alloc_fail label: free list, print internal error, return -1
}

```

A few notes to keep in mind while filling these in:

tokenlist_append and lex_line share an alloc_fail pattern in the original – decide if you want to keep the goto style or refactor.

read_word is the trickiest one; it's worth writing out the state transitions (plain char / escape / double-quote / single-quote) on paper before coding it.

Keep buf_push's growth strategy consistent (doubling) so you don't have subtle off-by-one bugs with the null terminator.

For this task {i pasted the parser part of the question}, I want you to give me a function skeleton code with todos

to help me write the code (make sure not to provide any code inside the functions)

```
#include <stdio.h>
#include "parser.h"

// return value = 1 (valid) / 0 (invalid)
int parse_validate(const TokenList *tokens){
    // TODO: start curr at tokens->head
    // TODO: empty input or immediate EOF => valid (return 1)
    // TODO: first token must be TOK_WORD, else return 0
    // TODO: advance curr past the first word

    // TODO: loop:
    //   - if curr is NULL or TOK_EOF => return 1 (end of valid sentence)
    //   - if curr is TOK_WORD => just advance (words can chain, e.g.
args)
    //   - if curr is TOK_LT / TOK_GT / TOK_GTGT (redirection):
    //       advance, then require next token to be TOK_WORD (filename),
    //       else return 0; then advance past that word
    //   - if curr is TOK_PIPE / TOK_SEMI:
    //       advance, then require next token to be TOK_WORD,
    //       else return 0; then advance past that word
    //   - if curr is TOK_AMP (background op):
    //       advance; if NULL or TOK_EOF => return 1 (valid trailing &)
    //       else require TOK_WORD, else return 0; then advance
    //   - anything else => return 0 (invalid token in this position)
}
```

Help me solve this task {i pasted B1} with skeleton functions and a todo. Make no mistakes

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>
#include <string.h>
#include <unistd.h>
```

```

#include <sys/stat.h>

#include "hop.h"

// IMPORTANT ASSUMPTION: hop() function takes in argc, *argv[]

#define STRING_SIZE 4096
char* home_directory;

typedef struct HopNode{
    char path[STRING_SIZE];
    int freq;
    long last_visit;
    struct HopNode *next;
}HopNode;

static HopNode *hopps = NULL;
static char previous_dir[STRING_SIZE] = ""; // initially none

static void get_history_path(char* buffer, int size_of_buffer){
    // TODO: if home_directory is NULL, set buffer[0] = '\0' and return
    // TODO: else snprintf "<home>/hop_history.txt" into buffer
}

static void load_history(void){
    // TODO: build history path via get_history_path; bail if empty
    // TODO: fopen for reading; bail if it doesn't exist yet
    // TODO: read line by line with fgets
    // - sscanf each line as "path\tfreq\tlast_visit"
    // (skip malformed lines)
    // - malloc a HopNode, copy path in (bounded), set freq/last_visit
    // - push onto the front of the `hopps` list
    // TODO: fclose when done
}

static HopNode *find_node(char* path){
    // TODO: walk `hopps`, strcmp each node's path against `path`
    // TODO: return the matching node, or NULL if none found
}

static HopNode *create_node(char *path){

```

```

    // TODO: malloc a HopNode, return NULL on failure
    // TODO: copy path in (bounded, null-terminated)
    // TODO: init freq = 1, last_visit = time(NULL), next = NULL
    // TODO: return the new node
}

static bool directory_exists(char* path){
    // TODO: stat() the path; return false if it fails
    // TODO: return true iff S_ISDIR on the resulting mode
}

static void record_visit(char* path){
    // TODO: look up path with find_node
    // TODO: if found: bump freq, refresh last_visit, return
    // TODO: if not found: create_node, push to front of `hopps`
    // (handle malloc failure gracefully)
}

static void save_history(void){
    // TODO: build history path via get_history_path; bail if empty
    // TODO: fopen for writing (truncates old history)
    // TODO: walk `hopps`, write one line per node:
    "path\tfreq\tlast_visit\n"
    // TODO: fclose
}

static int change_directory(char* target, char* home){
    // TODO: getcwd() to capture current dir before moving; bail (return
    0) on failure
    // TODO: chdir(target); bail (return 0) on failure
    // TODO: on success, save the pre-move dir into `previous_dir`
    // TODO: persist previous_dir to "<home>/prev_dir.txt" so it survives
    restarts
    // TODO: getcwd() again to get the new absolute path
    // TODO: record_visit() + save_history() on the new path
    // TODO: return 1 on success
}

static HopNode *find_best_match(char *name){
    // TODO: walk `hopps`, keep a running best (node + score)
    // TODO: only consider nodes whose path contains `name` as substring

```

```

(strstr)
    // TODO: skip candidates that fail directory_exists (stale entries)
    // TODO: compute a deterministic score combining freq and last_visit
    //          (e.g. weight freq heavily, add recency as tiebreaker)
    // TODO: track the highest-scoring valid candidate
    // TODO: return best match, or NULL if nothing qualifies
}

int hop(int argc, char* argv[], char* home_dir){
    static bool is_initialized = false;
    // TODO: stash home_dir into home_directory
    // TODO: on first call only, load_history() and flip is_initialized

    // TODO: if argc == 1 (no arguments): treat like "~"
    //   - change_directory to home_directory
    //   - print "hop: no such directory" on failure
    //   - return 0

    // TODO: otherwise loop over argv[1..argc-1], for each argument:
    //   - "~" => change_directory to home; on failure print error and
stop processing further args
    //   - "." => no-op, continue to next arg
    //   - ".." => change_directory(".."); on failure print error and stop
    //   - "-" => if no previous_dir recorded, skip (continue);
    //               else change_directory(previous_dir); on failure print
error and stop
    //   - anything else ("name"):
    //       1. try change_directory(argument) directly (relative/absolute
path)
    //       2. if that fails, find_best_match(argument) via frequency
    //           - if a match exists, change_directory to it (print error +
stop on failure)
    //           - if no match, print "hop: no such directory" and stop
processing further args
    // TODO: return 0
}

```

A few design points worth thinking through as you fill these in:

Scoring formula: the spec leaves this up to you, but it needs to be deterministic and blend frequency + recency (look at how zoxide's aging/decay works for inspiration – you don't have to copy it exactly).

Stale entries: find_best_match needs to skip directories that no longer exist and move to the next-best

candidate rather than just failing outright.

Persistence across sessions: `load_history` / `save_history` are your only touchpoints with disk – make sure the file format you choose round-trips cleanly (tab-separated fields is one simple option).

Where `previous_dir` gets set: only successful `change_directory` calls should update it, and it should be captured before the `chdir` happens.

Similarly help me solve {i pasted B2}. make no mistakes

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include <unistd.h>
#include <ctype.h>

#include "reveal.h"

#define MAX_PATH 4096
#define BIGGER_BUFFER (MAX_PATH * 2) // for paths that are might be longer
#define MAX_ENTRIES 4096

static int cmp(const void *a, const void *b){
    // TODO: compare two (char **) entries with strcmp for qsort ordering
}

static int read_prev_dir(const char* home_dir, char* out, int out_size){
    // TODO: build "<home_dir>/prev_dir.txt" path
    // TODO: fopen for reading; return 0 if it doesn't exist
    // TODO: fgets the first line; return 0 on failure
    // TODO: strip trailing \n / \r / whitespace
    // TODO: return 0 if the resulting line is empty
    // TODO: copy trimmed line into `out` (bounded), return 1
}

static void reveal_list_dir(const char *path, const char *display_prefix,
int show_all, int recursive) {
    // TODO: opendir(path); return early if it fails
    // TODO: readdir loop, collecting entry names into `names[]`
    // - skip "." and ".."
    // - skip dotfiles unless show_all is set
```

```

    // - strdup each kept name, bump count (respect MAX_ENTRIES)
    // TODO: closedir
    // TODO: qsort `names` lexicographically using cmp()
    // TODO: for each name, in order:
    // - build fullpath = path + "/" + name
    // - stat it to check if it's a directory
    // - print it, prefixed with display_prefix if non-empty,
    //   appending "/" if it's a directory
    // TODO: if recursive: for each name that is a directory,
    //   build the new display_prefix (prefix + "/" + name, or just name
    //   if prefix is empty) and recurse into reveal_list_dir on its
fullpath
    // TODO: free all the strdup'd names
}

static int resolve_reveal_path(const char* arg, char *resolved, int
resolved_size, const char* home_dir){
    // TODO: getcwd() into a local buffer; return -1 on failure
    // TODO: resolve `arg` into `resolved`:
    // - NULL or "." => current working directory
    // - "~"          => home_dir
    // - ".."         => cwd with the last path component stripped
    //                  (handle root "/" edge case)
    // - "-"          => read_prev_dir(home_dir, ...); fail (-1) if none
recorded
    // - starts with '/' => treat as absolute path as-is
    // - starts with "~/ " => home_dir + rest of the path
    // - otherwise       => cwd + "/" + arg (relative path)
    // TODO: stat `resolved`; return -1 if it doesn't exist or isn't a
directory
    // TODO: return 0 on success
}

static int reveal_is_flag_arg(const char *arg){
    // TODO: not a flag if it doesn't start with '-', or is just "-"
    // TODO: walk chars after '-'; if any char isn't 'a' or 't', not a
flag
    // TODO: otherwise it is a valid flag arg, return 1
}

void reveal_command(char **args, int argc, const char *home_dir){

```

```

// TODO: init reveal_all = 0, recursive = 0, path_arg = NULL,
//       path_arg_count = 0, seen_path_arg = 0, syntax_error = 0

// TODO: loop over args[1..argc-1]:
//   - if reveal_is_flag_arg(args[i]):
//       if a path arg was already seen, mark syntax_error (flags must
//       come before the path arg)
//       else scan the flag's characters, setting reveal_all /
recursive
//       for 'a' / 't' respectively
//   - else: treat as the path argument
//       bump path_arg_count, store it, set seen_path_arg = 1

// TODO: if syntax_error or more than one path arg was given,
//       print "reveal: invalid syntax" and return

// TODO: resolve_reveal_path(path_arg, ...);
//       on failure print "reveal: no such directory" and return

// TODO: reveal_list_dir(resolved, "", reveal_all, recursive)
}

```

A few things to be careful about when you implement this:

-t with no -a: recursion should still skip hidden subdirectories entirely (don't descend into them, and don't list their contents) unless -a is also set.

- before any hop: read_prev_dir needs to correctly return failure when prev_dir.txt doesn't exist yet – that's what produces the "no such directory" behavior required by the spec.

Repeated/combined flags (like the -ta -ttttt... example): since you're just OR-ing booleans as you scan characters, repeats should naturally have no extra effect – just make sure your flag-validation logic (reveal_is_flag_arg) doesn't choke on a long run of the same letter.

Ordering guarantee: qsort with cmp() needs to run before both the print loop and the recursive descent, so entries are alphabetical at every level.

Similarly give me the boilerplate code and todos for this {i pasted B3}. make no mistakes

```

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

```

```

#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <errno.h>

#include "peek.h"

#define CHUNK_SIZE 4096
#define INITIAL_LINE_CAP 128

typedef struct {
    char *data;
    int len;
    long orig_number; // non zero line number (index)
} PeekLine;

typedef struct {
    PeekLine *lines;
    int count;
    int cap;
} PeekLineList;

typedef struct { // ops = options
    bool numbered;
    bool reversed;
} PeekOptions;

static bool peek_is_flag_arg(const char *arg) {
    // TODO: not a flag if it doesn't start with '-', or is just "-"
    // TODO: walk chars after '-'; if any char isn't 'n' or 'r', not a
flag
    // TODO: otherwise return true
}

static int peek_parse_args(int argc, char *argv[], PeekOptions *ops, const
char **filenames_out) {
    // TODO: init ops->numbered = ops->reversed = false, fcount = 0
    // TODO: loop over argv[1..argc-1]:
    // - if peek_is_flag_arg: scan chars, set numbered/reversed
accordingly

```

```

    // - else if it starts with '-' but isn't a valid flag combo: return
-1
    //      (filenames starting with '-' aren't supported)
    // - else: store into filenames_out[fcount++]
    // TODO: return fcount
}

static void peek_linelist_init(PeekLineList *ll) {
    // TODO: zero out lines/count/cap
}

static void peek_linelist_push(PeekLineList *ll, char *data, int len) {
    // TODO: grow (realloc, doubling from INITIAL_LINE_CAP) if count ==
cap
    //      (exit(1) with an OOM message on failure)
    // TODO: store data/len into the next slot, orig_number = 0
    // TODO: increment count
}

static void peek_linelist_free(PeekLineList *ll) {
    // TODO: free each line's data
    // TODO: free the lines array itself
    // TODO: reset lines/count/cap
}

static bool peek_line_is_nonempty(const char *data, int len) {
    // TODO: false if len == 0
    // TODO: false if the line is just a lone '\n'
    // TODO: otherwise true
}

static void peek_linelist_assign_numbers(PeekLineList *ll, bool
stored_in_forward_order, long *running_no) {
    // TODO: if stored_in_forward_order: walk 0..count-1 in order,
    //      assigning ++(*running_no) to each non-empty line's
orig_number
    // TODO: else: walk count-1..0 (i.e. reverse index order),
    //      assigning ++(*running_no) to each non-empty line's
orig_number
    //      (this handles lines that were pushed in reverse-file-order)
}

```

```

static void peek_print_line(const PeekLine *ln, const PeekOptions *ops) {
    // TODO: if ops->numbered and orig_number > 0, printf the number +
space
    // TODO: fwrite the line's raw bytes to stdout
}

static void print_and_maybe_number(const char *data, int len, const
PeekOptions *ops, long *running_no) {
    // TODO: if numbered and line is non-empty, compute num = ++
(*running_no)
    // TODO: if numbered and num > 0, printf the number + space
    // TODO: fwrite the raw bytes to stdout
}

// Streams forward by printing each line as its read
static int peek_forward_fd(int fd, const PeekOptions *ops, long
*running_no) {
    // TODO: read in CHUNK_SIZE blocks in a loop
    // - on each block, scan for '\n' boundaries
    // - when a boundary is found: if there's leftover `pending` data
from
    // a previous partial line, glue it to this segment before
printing
    // (print_and_maybe_number), else print the segment directly
    // - carry any trailing partial line (after the last '\n') into
`pending`
    // via realloc, to be completed by the next read (or EOF)
    // - handle EINTR by retrying the read
    // TODO: after the loop, flush any remaining `pending` data as a final
line
    // TODO: return 0 on success, -1 on read error
}

// Reads one line (includes '\n') from fd into a malloc buffer
static char *read_line_fd(int fd, int *out_len, bool *got_any) {
    // TODO: read one byte at a time into a growable buffer (double cap as
needed)
    // TODO: set *got_any = true as soon as any byte is read
    // TODO: stop at '\n' (inclusive) or EOF
    // TODO: handle EINTR by retrying

```

```

    // TODO: if nothing was read at all, free buffer, return NULL
    // TODO: else set *out_len and return the buffer
}

// Reverse path for unseekable input: buffer fully, then print back to
front
static int peek_reverse_buffered_fd(int fd, PeekLineList *ll, long
*running_no) {
    // TODO: loop calling read_line_fd, pushing each line via
peek_linelist_push
    //      (stored in forward order) until got_any is false
    // TODO: assign line numbers in forward order
(peek_linelist_assign_numbers, true)
    // TODO: return 0
}

// Walk backward in fixed-size lseek/read chunks rather than loading the
whole file into memory
static int peek_reverse_seek_fd(int fd, PeekLineList *ll, long
*running_no) {
    // TODO: lseek to SEEK_END to get file_size; if that fails, rewind and
    //      fall back to peek_reverse_buffered_fd
    // TODO: if file_size == 0, return 0 (nothing to do)
    // TODO: walk backward from file_size in CHUNK_SIZE steps:
    //   - lseek to the chunk's start position, read `to_read` bytes
    //   - glue with any `tail` bytes carried over from the previous
    //     (further-right) chunk into `combined`
    //   - scan `combined` backward for '\n' boundaries, pushing each
    //     complete line found onto `ll` (these come out in reverse order)
    //   - when reaching pos == 0, flush whatever's left at the start of
    //     `combined` as the final (first) line
    //   - otherwise stash the remaining prefix as the new `tail` for the
    //     next iteration further left
    //   - handle EINTR on read, free combined/tail appropriately
    // TODO: assign line numbers bottom-to-top
(peek_linelist_assign_numbers, false)
    // TODO: return 0 on success, -1 on error
}

static bool peek_fd_is_seekable(int fd) {
    // TODO: return true iff lseek(fd, 0, SEEK_CUR) succeeds

```

```

}

static int peek_process_source(const char *name, bool is_stdin, const
PeekOptions *ops, long *running_no) {
    // TODO: resolve fd:
    //   - if is_stdin: fd = STDIN_FILENO
    //   - else: stat(name) first
    //       - missing => print "peek: no such file or directory", return
-1
    //       - is a directory => print "peek: is a directory", return -1
    //       - open(name, O_RDONLY); fail => print "no such file or
directory", return -1
    // TODO: if ops->reversed:
    //   - init a PeekLineList
    //   - if not stdin and peek_fd_is_seekable: use peek_reverse_seek_fd,
    //       then print lines in the order they were pushed (already correct
order)
    //   - else: use peek_reverse_buffered_fd, then print lines back-to-
front
    //   - free the line list
    // TODO: else: peek_forward_fd
    // TODO: close fd if not stdin
    // TODO: return the resulting rc
}

int peek_command(int argc, char *argv[]) {
    // TODO: malloc a filenames array sized to argc
    // TODO: peek_parse_args(); on -1 print "peek: invalid syntax",
cleanup, return 1
    // TODO: if fcount == 0: process a single stdin source
    // TODO: else: loop over each filename
    //   - treat "-" as stdin
    //   - call peek_process_source for each, tracking any_error
    //   - running_no persists (is shared) across files so numbering
    //       continues across concatenated files
    // TODO: free filenames array
    // TODO: return 1 if any_error else 0
}

```

A few subtleties worth flagging as you implement this:

Numbering across files: `running_no` is passed by pointer and threaded through every call, so line numbers keep incrementing across multiple files rather than resetting per file — that's what the last example in the spec relies on.

`-r` with multiple files: each file's lines are reversed independently, but overall file order on the command line stays the same — so the outer loop in `peek_command` must not itself reverse anything, only `peek_process_source` per file.

Seekable vs buffered reverse paths produce lines in different push-orders (`peek_reverse_seek_fd` pushes back-to-front, `peek_reverse_buffered_fd` pushes front-to-back) — make sure the corresponding print loops in `peek_process_source` match each one's storage order, or output will come out backwards or forwards incorrectly.

Partial final line without trailing `\n`: both `peek_forward_fd` and the line-list based paths need to treat a trailing chunk with no newline as its own valid last line.

Similarly give me the boilerplate code and todos for this {i pasted B4}. make no mistakes

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

#include "locate.h"

int is_executable(const char* path){
    // TODO: stat() the path; return 0 if it fails
    // TODO: return 0 if it isn't a regular file (S_ISREG)
    // TODO: return 0 if access(path, X_OK) fails (not executable)
    // TODO: otherwise return 1
}

void locate(char **args, int argc) {
    // TODO: if argc < 2 (no filenames given), print "locate: invalid
syntax", return

    // TODO: fetch PATH via getenv; fall back to "" if unset

    // TODO: loop over args[1..argc-1], for each `filename`:
    // - track a `found` flag for this arg
```

```

//      - check cwd first:
//          getcwd() to get the current directory
//          build full_path = cwd + "/" + filename
//          if is_executable(full_path): print it, mark found

//      - then walk PATH entries:
//          strdup the PATH string (strtok mutates it)
//          strtok on ':' to split into directories, in order
//          for each dir: build full_path = dir + "/" + filename
//              if is_executable(full_path): print it, mark found
//          free the strdup'd copy

//      - if never found across cwd + all PATH entries:
//          print "locate: command not found (filename)"
}

```

A couple of things worth double-checking as you fill this in:

Ordering matters: the cwd check must run and print before the PATH loop starts, since the spec requires cwd matches to appear first.

Multiple matches are all printed: don't break or return early once you find one match in a directory list — a name can legitimately appear in several PATH entries (see the sudo example), and each occurrence prints its own line.

strtok mutates its input, so working on a strdup'd copy of PATH (rather than the original getenv result) avoids corrupting it across the outer for loop's iterations.

Similarly give me the boilerplate code and todos for this task {C1}

NOTE: This code was later modified and used for redirect.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>
#include <sys/wait.h>

#include "exec.h"

```

```

// checks if a path is a runnable regular file
int is_exec(const char *path) {
    // TODO: stat() the path; return 0 if it fails
    // TODO: return 0 if it isn't a regular file (S_ISREG)
    // TODO: return 0 if access(path, X_OK) fails
    // TODO: otherwise return 1
}

// resolves cmd to a full path following the rules
// returns a malloc string on success, NULL if nothing found
char *resolve_command(const char *cmd) {
    // TODO: if cmd contains a '/', treat as literal path:
    //         return strdup(cmd) if is_exec(cmd), else NULL (no searching)

    // TODO: if cmd starts with '%', set skip_cwd flag and advance past it

    // TODO: unless skip_cwd, check cwd first:
    //         getcwd(), build cwd + "/" + cmd, return strdup if is_exec

    // TODO: fall back to PATH:
    //         getenv("PATH"); return NULL if unset
    //         strdup the PATH string (strtok mutates it)
    //         strtok on ':' to walk each directory
    //         build dir + "/" + cmd, return strdup(full_path) if is_exec
    //         (free the path copy before returning)
    //         free the path copy if nothing found
    // TODO: return NULL if nothing matched anywhere
}

// runs a command with its args (args[0] = command, NULL terminated)
void execute_command(char **args) {
    // TODO: resolve_command(args[0]); if NULL, print
    //         "cshell: command not found (args[0])" and return

    // TODO: fork()
    // - child: execv(resolved, args); on failure perror + exit(1)
    // - parent: waitpid on the child, discard/inspect status as needed
    // - fork failure: perror

    // TODO: free the resolved path (in the parent, after waitpid)
}

```

A couple of things worth double-checking as you fill this in:

Order of checks matters: / in the name short-circuits everything else (no % handling, no PATH search) — literal paths are checked as-is and nothing more.

% stripping: make sure you're advancing the pointer past % before doing any path-building, but still using the original cmd[0] to detect it — don't mutate the caller's string in place if args[0] might be reused elsewhere.

Freeing resolved: since resolve_command always returns a strdup'd string on success, both the "command not found" path and the success path need to be checked against a free, otherwise you'll leak one allocation per invocation.

waitpid status: you're not currently doing anything with status beyond reaping the child — depending on how the rest of your shell reports exit codes, you may want to inspect WIFEXITED/WEXITSTATUS here later.

I now have to do this task {C2}, for doing this I want to just make changes to the exec.c{i pasted my version of exec.c} give me similar boilerplate functions and todos to satisfy this task

```
static int is_exec(const char *path) {
    // TODO: stat() the path; return 0 if it fails
    // TODO: return 0 if it isn't a regular file (S_ISREG)
    // TODO: return 0 if access(path, X_OK) fails
    // TODO: otherwise return 1
}

// generates a full path out of the given command
static char *resolve_command(const char *cmd) {
    // TODO: needed so execute_with_redirection has something to execv();
    //      reuse your Part C1 resolve_command logic unchanged here
}

// writes all n bytes in buf to fd, looping through partial writes
// returns 0 on success, -1 on error
static int write_all(int fd, const char *buf, int n) {
    // TODO: loop calling write() until `n` bytes are written
    // TODO: retry on EINTR
    // TODO: return -1 on any other error, 0 once fully written
    //      (needed by setup_input_redirection to stream file contents
```

```

//          into the pipe without losing data on partial writes)
}

// opens every input file in order and streams contents into a pipe so th
// multiple "< f1 < f2" behave like one continuous stdin
// returns read end of pipe, or -1 on error
static int setup_input_redirection(char **input_files, int input_count,
                                   int *writer_pid_out,
                                   int (*pipefds)[2], int num_pipes) {
    // TODO: create a pipe(); on failure perror + return -1
    // TODO: open() every file in input_files with O_RDONLY, in order
    //      - if any open() fails: print "cshell: no such file or direct
    //      close everything already opened (including the pipe), retu
    // TODO: fork() a helper process:
    //      - child:
    //          close the pipe's read end
    //          close all pipefds[0..num_pipes-1] it inherited (not its conc
    //          for each opened input fd, in order:
    //              read in chunks, write_all() each chunk into the pipe's wri
    //              close that input fd once fully read
    //          close the pipe's write end, exit(0)
    //      - parent:
    //          close all the input file fds (child owns its own copies)
    //          close the pipe's write end (parent only needs the read end)
    //          store the child's pid into *writer_pid_out
    //          return the pipe's read end
}

// runs args[0] with input files only (no output redirection)
void execute_with_redirection(char **args, char **input_files, int input_
    // TODO: resolve_command(args[0]); if NULL, print
    //      "cshell: command not found (args[0])" and return

    // TODO: if input_count > 0:
    //      call setup_input_redirection(); if it returns -1, free resol
    //      and return (error already printed)

    // TODO: fork()
    //      - child: if in_fd was set, dup2(in_fd, STDIN_FILENO) then close
    //              execv(resolved, args); perror + exit(1) on failure
    //      - parent: close in_fd if set (child has its own copy)
    //                  waitpid on the main child
    //                  waitpid on writer_pid if it was set (reap the streamir
    //      - fork failure: perror, still close in_fd / reap writer_pid if s

```

```

    // TODO: free(resolved)
}

```

Multiple `< f1 < f2` must concatenate, not overwrite – that's the whole reason `setup_input_redirection` uses a pipe-and-helper-process pattern instead of just `dup2`-ing one file directly: a single `dup2` can only point `stdin` at one `fd`, so a separate writer process is needed to stream several files into it as one stream. Missing file must abort before `exec` – `open()` failures need to be caught in `setup_input_redirection` itself, before any `fork` of the actual command happens, so `cat` never even attempts to run.

Closing fds you don't need – both the helper child and the main parent close file descriptors that only the other process needs (input file fds in the parent, the pipe write-end in the parent, etc.) to avoid fd leaks and to make sure EOF propagates correctly once writing is done.

Reaping the helper process – `writer_pid` must be `waitpid`'d separately from the main command's pid, or it'll linger as a zombie.

Now give me the skeleton functions and todos for {C3} that i can add to my code {pasted redirect.c}.

```
// opens every output file with the right mode (truncate or append).
// returns 0 on success (fds filled in out_fds), -1 on error (nothing left
open)

static int open_output_files(char **output_files, int *append_flags,
                             int output_count, int *out_fds) {
    // TODO: loop over each output file
    //   - build flags: O_WRONLY | O_CREAT | (append_flags[i] ? O_APPEND :
O_TRUNC)
    //   - open(output_files[i], flags, 0644)
    //   - on failure: print "cshell: unable to create file for writing",
    //     close every fd opened so far in this loop, return -1
    // TODO: return 0 if all opened successfully
}
```

[illegible]

```

    // TODO: open_output_files(); if it fails, return -1 (error already
printed)
    // TODO: create a pipe(); on failure perror, close all out_fds, return
-1
    // TODO: fork() a helper process:
    //   - child:
    //       close the pipe's write end
    //       close all pipefds[0..num_pipes-1] it inherited (not its
concern)
    //       loop: read() from the pipe's read end into a buffer
    //       for each output fd, write_all() the same chunk to it
    //       close the pipe's read end, close every out_fd, exit(0)
    //   - parent:
    //       close all the out_fds (child owns its own copies)
    //       close the pipe's read end (parent only needs the write end)
    //       store the child's pid into *reader_pid_out
    //       return the pipe's write end - this is what the command's
    //       stdout gets dup2'd onto
}

// runs args[0] with input files and output files
void execute_with_redirection(char **args,
                             char **input_files, int input_count,
                             char **output_files, int *append_flags, int
output_count) {
    // TODO: resolve_command(args[0]); if NULL, print
    //       "cshell: command not found (args[0])" and return

    // TODO: if input_count > 0: setup_input_redirection() as before (C2);
    //       on failure free resolved and return

    // TODO: if output_count > 0: setup_output_redirection();
    //       on failure: close in_fd if set, waitpid writer_pid if set,
    //       free resolved, and return (error already printed)

    // TODO: fork()
    //   - child:
    //       if in_fd set: dup2(in_fd, STDIN_FILENO), close(in_fd)
    //       if out_fd set: dup2(out_fd, STDOUT_FILENO), close(out_fd)
    //       execv(resolved, args); perror + exit(1) on failure
    //   - parent:

```

```

//      close in_fd / out_fd if set (child has its own copies)
//      waitpid the main child
//      waitpid writer_pid if set, reader_pid if set
//  - fork failure: perror, still clean up in_fd/out_fd and reap
//      writer_pid/reader_pid if they were set

// TODO: free(resolved)
}

```

Independent modes per file: `open_output_files` must look at `append_flags[i]` per file, since `> f1 >> f2` mixes truncate and append in the same command – one shared flag would be wrong.

Fan-out to multiple files: like input redirection, a single `dup2` can only point stdout at one fd, so `setup_output_redirection` uses the same pipe-plus-helper-process trick, but in reverse – a reader process pulls from the pipe and `write_all()`s the same bytes to every listed output file.

Terminal must see nothing: since the command's stdout is `dup2`'d straight onto the pipe's write end (not left attached to the real terminal at all), output only reaches the terminal if you didn't set up a redirection – this falls out naturally as long as you only touch stdout when `out_fd != -1`.

Fail before executing: if any output file can't be opened, `open_output_files` must catch it and bail before `execute_with_redirection` forks the actual command – matching the "not execute the command" requirement.

Reap both helpers: with both redirections active you now have up to two extra child processes (`writer_pid` for input, `reader_pid` for output) that need separate `waitpid` calls, on top of the main command's pid.

Now i need to extend my code so that piping functions work look at this task {C4}. Give me the similar boilerplate functions and the todos which will help me solve the task.

```

// closes both ends of every pipe in pipefds[0..num_pipes-1]
static void close_all_pipes(int (*pipefds)[2], int num_pipes) {
    // TODO: loop over each pipe, close its read end if != -1, close its
    //      write end if != -1
    //      (needed because children and the parent both need a single
    //      cleanup pass over the whole pipeline's pipes, and by the time
    //      cleanup happens some individual fds may already be -1)
}

```

```

// runs a full pipeline of commands, wiring stdout of stage i to stdin of

```

```

// stage i+1 via pipe(), while still honoring any per-stage file
redirection
// (e.g. "cmd1 < in.txt | cmd2 > out.txt")
void execute_pipeline(command_stage *stages, int num_stages) {
    // TODO: if num_stages <= 0, return

    // TODO: if num_stages == 1, this is just a normal redirected command
-
    //      delegate straight to execute_with_redirection() and return
    //      (no pipeline machinery needed for a single stage)

    // TODO: num_pipes = num_stages - 1; declare pipefds[num_pipes][2],
    //      init every slot to -1

    // TODO: 1. create every pipe up front, before touching any stage
    //      - pipe(pipefds[i]) for i in 0..num_pipes-1
    //      - on any failure: perror, close_all_pipes(), return

    // TODO: declare per-stage bookkeeping arrays, all sized num_stages:
    //      resolved[] (char*), writer_pid[]/reader_pid[] (from
input/output
    //      redirection helpers), in_fd[]/out_fd[] (extra fds from file
    //      redirection), child_pid[] - init all to NULL/-1

    // TODO: 2. for each stage, in the parent, BEFORE forking any pipeline
    //      children:
    //      - if stage has input_count > 0: setup_input_redirection(),
    //      passing pipefds/num_pipes so the helper closes them too;
    //      on failure mark aborted and break
    //      - if stage has output_count > 0: setup_output_redirection(),
    //      same pipefds passing; on failure mark aborted and break
    //      - resolve_command(stage.args[0]); if NULL, print
    //      "cshell: command not found (name)" for this stage but do
    //      NOT abort the whole pipeline - just leave resolved[i] NULL
    //      so this stage gets skipped later (rest of pipeline still
runs)
    //      (doing resolution/file-redirection setup entirely in the
parent,
    //      before any fork, means errors print to the real terminal
instead
    //      of vanishing into a pipe)

```

```

// TODO: if aborted (a file/pipe error, not a "command not found"):
//      clean up every stage's resolved/in_fd/out_fd, waitpid any
//      writer_pid/reader_pid already spawned, close_all_pipes(),
return

// TODO: 3. fork one child per stage whose resolved[i] != NULL:
//      (skip forking entirely for a stage that failed to resolve –
//      its pipe fds simply get closed by the parent's normal
cleanup,
//      which sends EOF downstream correctly)
//      - child:
//          stdin: file redirection (in_fd[i]) takes priority; else
//              if i > 0, dup2 pipefds[i-1][0] (read end of the
//              previous pipe); else inherit real stdin
//          stdout: file redirection (out_fd[i]) takes priority; else
//              if i < num_stages-1, dup2 pipefds[i][1] (write end
//              of this stage's own pipe); else inherit real
stdout
//          close_all_pipes() – the child doesn't need any pipe fds
//          after its dup2 calls
//          close in_fd[i]/out_fd[i] if they were used (already
dup2'd)
//          execv(resolved[i], stage.args); perror + exit(1) on
failure
//      - parent: store the pid into child_pid[i]
//      - fork failure: perror, leave child_pid[i] as -1

// TODO: 4. parent closes every pipe fd it holds via
close_all_pipes(),
//      plus every in_fd[i]/out_fd[i] it still holds – now that every
//      forked child has its own copies

// TODO: 5. wait for every stage's child_pid[i] (if > 0), plus every
//      writer_pid[i]/reader_pid[i] (if > 0), plus free every
//      resolved[i] that's non-NULL
}

```

Key things to get right this task specifically:

Order of setup matters: resolving commands and setting up file redirection before forking pipeline children means a bad command name or missing file prints cleanly to the terminal, not into a half-wired pipe – and it lets you decide which stages to fork before you fork any of them.

A failed stage doesn't kill the pipeline: per requirement, command not found for one stage must not stop the rest — this only works cleanly if you simply skip forking for that stage and let its neighbors' pipe ends close normally (which delivers EOF to the downstream reader exactly as if that stage had run and produced no output).

stdin/stdout priority per stage: file redirection always wins over the pipe connection for that stage — so `cmd1 < in.txt | cmd2 > out.txt` needs `in_fd/out_fd` checked before falling back to `pipefds[i-1][0]/pipefds[i][1]`.

Every child must close pipes it isn't using: forgetting this is the classic pipeline bug — a lingering write-end fd in some unrelated child keeps the pipe "open" from the reader's perspective, so sort (or whatever's downstream) never sees EOF and hangs forever.

Reaping helpers, not just pipeline children: with input/output redirection combined with piping, you can have a `writer_pid/reader_pid` helper per stage, on top of that stage's own `child_pid` — all of them need `waitpid`, or you'll leak zombies.

Help me with my make file, i have files in this format {I have uploaded the pic of the dir}. make sure the format is of the given makefile {Given makefile that was there on website}. Also explain the makefile code.

Given that your directory looks like this:

```
project/
├─ Makefile
├─ include/
│   ├─ lexer.h
│   ├─ parser.h
│   ├─ hop.h
│   ├─ reveal.h
│   ├─ peek.h
│   ├─ locate.h
│   └─ exec.h
└─ redirect.h
├─ src/
│   ├─ lexer.c
│   ├─ parser.c
│   ├─ hop.c
│   ├─ reveal.c
│   ├─ peek.c
│   └─ locate.c
```

```
└─ exec.c
└─ redirect.c
```

Here is the Makefile:

```
CC := gcc
CFLAGS := -std=c23 -D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 \
          -Wall -Wextra -Werror -Wno-unused-parameter -fno-asm \
          -Iinclude
LDFLAGS :=

SRC_DIR := src
BUILD_DIR := build
TARGET := shell.out

SRCS := $(wildcard $(SRC_DIR)/*.c)
OBJS := $(patsubst $(SRC_DIR)/%.c,$(BUILD_DIR)/%.o,$(SRCS))

.PHONY: all clean

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(OBJS) -o $@ $(LDFLAGS)

$(BUILD_DIR)/%.o: $(SRC_DIR)/%.c | $(BUILD_DIR)
    $(CC) $(CFLAGS) -c $< -o $@

$(BUILD_DIR):
    mkdir -p $(BUILD_DIR)

clean:
    rm -rf $(BUILD_DIR) $(TARGET)
```

Explanation, line by line / section by section:

Variables (:= is immediate assignment – evaluated once, when the line is read):

CC := gcc – the compiler to use. Referenced later as \$(CC).

CFLAGS – compiler flags applied to every .c file compiled:

-std=c23 pins the C standard version.

-D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 expose POSIX/XOPEN functions (getcwd, lseek, fork, etc. – exactly what your hop.c/peek.c/redirect.c rely on) that -std=c23 alone would

otherwise hide.

-Wall -Wextra -Werror turn on strict warnings and treat them as errors, forcing clean code.

-Wno-unused-parameter silences one specific warning class (useful since many of your functions take argc/argv-style parameters they don't all use).

-fno-asm disables inline-assembly extensions.

-Iinclude tells the compiler to look in include/ for header files (so #include "lexer.h" resolves correctly from files in src/).

The trailing \ lets the flag list continue onto the next line — it's just line-continuation, not part of the flags.

LDFLAGS := — linker flags; currently empty, but there in case you later need to link against a library (e.g. -lm).

SRC_DIR, BUILD_DIR, TARGET — just path/name constants: source folder, output-object folder, and the final binary name.

Deriving file lists automatically:

SRCS := \$(wildcard \$(SRC_DIR)/*.c) — expands to every .c file in src/ (so src/hop.c, src/lexer.c, etc. are picked up automatically — you never list them by hand).

OBJS := \$(patsubst \$(SRC_DIR)/%.c,\$(BUILD_DIR)/%.o,\$(SRCS)) — for every source file, generates the path of its corresponding object file, swapping src/ for build/ and .c for .o. E.g. src/hop.c → build/hop.o.

.PHONY: all clean

Declares all and clean as "phony" targets — meaning they're just command labels, not actual files to check timestamps on. Without this, if a file named clean ever existed in the directory, make clean would do nothing (thinking it's already up to date).

Build rules:

all: \$(TARGET) — the default target when you just run make; it depends on (and thus triggers building) the final binary.

\$(TARGET): \$(OBJS) — the binary depends on all object files. The recipe \$(CC) \$(OBJS) -o \$@ \$(LDFLAGS) links them together. \$@ is an automatic variable meaning "the current target's name" (shell.out here).

\$(BUILD_DIR)/%.o: \$(SRC_DIR)/%.c | \$(BUILD_DIR) — a pattern rule: any build/X.o depends on the matching src/X.c. The | \$(BUILD_DIR) is an order-only prerequisite — it ensures the build/ directory exists first, but doesn't trigger a rebuild just because the directory's timestamp changed.

Recipe: \$(CC) \$(CFLAGS) -c \$< -o \$@ compiles one .c into one .o. \$< is "the first prerequisite" (the .c file), \$@ is "the target" (the .o file).

\$(BUILD_DIR): mkdir -p \$(BUILD_DIR) — creates the build/ directory if it doesn't exist (-p avoids errors if it's already there).

clean:

Removes the build/ directory and the compiled binary entirely — rm -rf \$(BUILD_DIR) \$(TARGET) — so you can start a fresh build with make clean && make.

With this in place, make alone builds shell.out, and adding a new file to src/+include/ (like a future background.c) requires zero Makefile edits — \$(wildcard ...) picks it up automatically.